

Concurrency Control

one of the fundamental properties of a transaction is isolation.

When several transactions execute concurrently in the database, however, the isolation property may no longer be preserved.

To ensure that it is, the system must control the interaction among the concurrent transactions; this control is achieved through one of a variety of mechanisms called concurrency control schemes.

As we shall see, there are a variety of concurrency-control schemes. No one scheme is clearly the best; each one has advantages.

the most frequently used schemes are two-phase locking and snapshot isolation.

Lock-Based Protocols

Screen clipping taken: 12-06-2026 08:50

while one transaction is accessing a data item, no other transaction can modify that data item.

The most common method used to implement this requirement is to allow a transaction to access a data item only if it is currently holding a **lock** on that item.

There are various modes in which a data item may be locked

1. **Shared.** If a transaction T_i has obtained a **shared-mode lock** (denoted by S) on item Q , then T_i can read, but cannot write, Q .
2. **Exclusive.** If a transaction T_i has obtained an **exclusive-mode lock** (denoted by X) on item Q , then T_i can both read and write Q .

Screen clipping taken: 12-06-2026 09:00

	S	X
S	true	false
X	false	false

Figure 15.1 Lock-compatibility matrix comp.

Screen clipping taken: 12-06-2026 09:01

1. We require that every transaction request a lock in an appropriate mode on data item Q , depending on the types of operations that it will perform on Q .
2. The transaction makes the request to the concurrency-control manager. The transaction can proceed with the operation only after the concurrency-control manager grants the lock to the transaction.
3. The use of these two lock modes allows multiple transactions to read a data item but limits write access to just one transaction at a time.
4. Notethatsharedmodeiscompatiblewithsharedmode,butnotwithexclusive mode. At any time, several shared-mode locks can be held simultaneously (by different transactions) on a particular data item.
5. A subsequent exclusive-mode lock request has to wait until the currently held shared-mode locks are released.
6. A transaction requests a shared lock on data item Q by executing the lock S(Q) instruction.
7. a transaction requests an exclusive lock through the lock-X(Q) instruction.
8. A transaction can unlock a data item Q by the unlock(Q) instruction.

To access a data item, transaction T_i must first lock that item. If the data item is already locked by another transaction in an incompatible mode, the concurrency control manager will not grant the lock until all incompatible locks held by other transactions have been released. Thus, T_i is made to wait until all incompatible locks held by other transactions have been released.

Transaction T_i may unlock a data item that it had locked at some earlier point

Unfortunately, locking can lead to an undesirable situation.

Consider the partial schedule of Figure 15.7 for T3 and T4.

Since T3 is holding an exclusive mode lock on B and T4 is requesting a shared-mode lock on B, T4 is waiting for T3 to unlock B.

Similarly, since T4 is holding a shared-mode lock on A and T3 is requesting an exclusive-mode lock on A, T3 is waiting for T4 to unlock A.

Thus, we have arrived at a state where neither of these transactions can ever proceed with its normal execution.

This situation is called **deadlock**. When deadlock occurs, the system must roll back one of the two transactions. Once a transaction has been rolled back, the data items that were locked by that transaction are unlocked. These data items are then available to the other transaction, which can continue with its execution.

15.1.3 The Two-Phase Locking Protocol

One protocol that ensures serializability is the **two-phase locking protocol**. This protocol requires that each transaction issue lock and unlock requests in two phases:

1. **Growing phase.** A transaction may obtain locks, but may not release any lock.
2. **Shrinking phase.** A transaction may release locks, but may not obtain any new locks.

Screen clipping taken: 12-06-2026 09:22

Initially, a transaction is in the growing phase. The transaction acquires locks as needed. Once the transaction releases a lock, it enters the shrinking phase, and it can issue no more lock requests.

Deadlock Handling

Screen clipping taken: 12-06-2026 09:28

1. A system is in a deadlock state if there exists a set of transactions such that every transaction in the set is waiting for another transaction in the set.
2. there exists a set of waiting transactions $\{T_0, T_1, \dots, T_n\}$ such that T_0 is waiting for a data item that T_1 holds,
3. T_1 is waiting for a data item that T_2 holds,
4. T_{n-1} is waiting for a data item that T_n holds, and T_n is waiting for a data item that T_0 holds.
5. None of the transactions can make progress in such a situation

The only remedy to this undesirable situation is for the system to invoke some drastic action, such as rolling back some of the transactions involved in the deadlock.

Rollback of a transaction may be partial: That is, a transaction may be rolled back to the point where it obtained a lock whose release resolves the deadlock.

There are two principal methods for dealing with the deadlock problem.

1. Deadlock prevention
 2. deadlock detection and deadlock recovery
-
1. We can use a deadlock prevention protocol to ensure that the system will never enter a deadlock state.
 2. we can allow the system to enter a deadlock state, and then try to recover by using a deadlock detection and deadlock recovery scheme.

Deadlock Prevention:

There are two approaches to deadlock prevention.

One approach ensures that no cyclic waits can occur by ordering the requests for locks, or requiring all locks to be acquired together.

The other approach is closer to deadlock recovery, and performs transaction rollback instead of waiting for a lock, whenever the wait could potentially result in a deadlock.

The simplest scheme under the first approach requires that each transaction locks all its data items before it begins execution. Moreover, either all are locked in one step or none are locked.

There are two main disadvantages to this protocol:

- (1) it is often hard to predict, before the transaction begins, what data items need to be locked;
- (2) data-item utilization may be very low, since many of the data items may be locked but unused for a long time.

The second approach for preventing deadlocks is to use preemption and transaction rollbacks.

The system uses these timestamps only to decide whether a transaction should wait or roll back. Locking is still used for concurrency control.

1. The **wait-die** scheme is a nonpreemptive technique. When transaction T_i requests a data item currently held by T_j , T_i is allowed to wait only if it has a timestamp smaller than that of T_j (that is, T_i is older than T_j). Otherwise, T_i is rolled back (dies).

For example, suppose that transactions T_{14} , T_{15} , and T_{16} have timestamps 5, 10, and 15, respectively. If T_{14} requests a data item held by T_{15} , then T_{14} will wait. If T_{24} requests a data item held by T_{15} , then T_{16} will be rolled back.

2. The **wound-wait** scheme is a preemptive technique. It is a counterpart to the wait-die scheme. When transaction T_i requests a data item currently held by T_j , T_i is allowed to wait only if it has a timestamp larger than that of T_j (that is, T_i is younger than T_j). Otherwise, T_j is rolled back (T_j is *wounded* by T_i).

Screen clipping taken: 12-06-2026 10:54

The major problem with both of these schemes is that unnecessary rollbacks may occur.

Another simple approach to deadlock prevention is based on **lock timeouts**.

In this approach, a transaction that has requested a lock waits for at most a specified amount of time.

If the lock has not been granted within that time, the transaction is said to time out, and it rolls itself back and restarts.

Deadlock Detection and Recovery:

a detection and recovery scheme must be used. An algorithm that examines the state of the system is invoked periodically to determine whether a deadlock has occurred. If one has, then the system must attempt to recover from the deadlock. To do so, the system must:

- Maintain information about the current allocation of data items to transactions, as well as any outstanding data item requests.
- Provide an algorithm that uses this information to determine whether the system has entered a deadlock state.
- Recover from the deadlock when the detection algorithm determines that a deadlock exists.

Screen clipping taken: 12-06-2026 11:12

15.2.2.1 Deadlock Detection

Deadlocks can be described precisely in terms of a directed graph called a **wait-for graph**. This graph consists of a pair $G = (V, E)$, where V is a set of vertices and E is a set of edges. The set of vertices consists of all the transactions in the system. Each element in the set E of edges is an ordered pair $T_i \rightarrow T_j$. If $T_i \rightarrow T_j$ is in E ,

then there is a directed edge from transaction T_i to T_j , implying that transaction T_i is waiting for transaction T_j to release a data item that it needs.

When transaction T_i requests a data item currently being held by transaction T_j , then the edge $T_i \rightarrow T_j$ is inserted in the wait-for graph. This edge is removed only when transaction T_j is no longer holding a data item needed by transaction T_i .

Screen clipping taken: 12-06-2026 11:22

A deadlock exists in the system if and only if the wait-for graph contains a cycle. Each transaction involved in the cycle is said to be deadlocked.

To detect deadlocks, the system needs to maintain the wait-for graph, and periodically to invoke an algorithm that searches for a cycle in the graph.

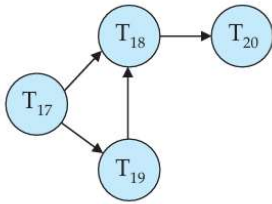


Figure 15.13 Wait-for graph with no cycle.

Screen clipping taken: 12-06-2026 11:24

To illustrate these concepts, consider the wait-for graph in Figure 15.13, which depicts the following situation:

- Transaction T_{17} is waiting for transactions T_{18} and T_{19} .
- Transaction T_{19} is waiting for transaction T_{18} .
- Transaction T_{18} is waiting for transaction T_{20} .

Since the graph has no cycle, the system is not in a deadlock state.

Screen clipping taken: 12-06-2026 11:24

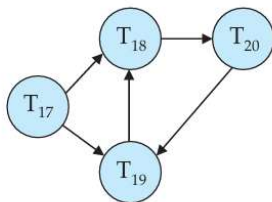


Figure 15.14 Wait-for graph with a cycle.

Screen clipping taken: 12-06-2026 11:27

Suppose now that transaction T_{20} is requesting an item held by T_{19} . The edge $T_{20} \rightarrow T_{19}$ is added to the wait-for graph, resulting in the new system state in Figure 15.14. This time, the graph contains the cycle:

$$T_{18} \rightarrow T_{20} \rightarrow T_{19} \rightarrow T_{18}$$

implying that transactions T_{18} , T_{19} , and T_{20} are all deadlocked.

Consequently, the question arises: When should we invoke the detection algorithm? The answer depends on two factors:

1. How often does a deadlock occur?
2. How many transactions will be affected by the deadlock?

If deadlocks occur frequently, then the detection algorithm should be invoked more frequently. Data items allocated to deadlocked transactions will be

Screen clipping taken: 12-06-2026 11:27

Recovery from Deadlock:

When a detection algorithm determines that a deadlock exists, the system must recover from the deadlock. The most common solution is to rollback one or more transactions to break the deadlock. Three actions need to be taken:

1. Selection of a victim.

2. Rollback.
3. Starvation

1. Selection of a victim.

Given a set of deadlocked transactions, we must determine which transaction (or transactions) to roll back to break the deadlock. We should roll back those transactions that will incur the minimum cost.

- a. How long the transaction has computed, and how much longer the transaction will compute before it completes its designated task.
- b. How many data items the transaction has used.
- c. How many more data items the transaction needs for it to complete.
- d. How many transactions will be involved in the rollback.

Screen clipping taken: 12-06-2026 11:37

2. Rollback. Once we have decided that a particular transaction must be rolled back, we must determine how far this transaction should be rolled back. The simplest solution is a total rollback: Abort the transaction and then restart it. However, it is more effective to roll back the transaction only as far as necessary to break the deadlock. Such partial rollback requires the system to maintain additional information about the state of all the running transactions.

3. Starvation. In a system where the selection of victims is based primarily on cost factors, it may happen that the same transaction is always picked as a victim. As a result, this transaction never completes its designated task, thus there is **starvation**. We must ensure that a transaction can be picked as a victim only a (small) finite number of times. The most common solution is to include the number of rollbacks in the cost factor.

Screen clipping taken: 12-06-2026 11:40

Multiple Granularity

Screen clipping taken: 12-06-2026 12:48

What is needed is a mechanism to allow the system to define multiple levels of granularity.

This is done by allowing data items to be of various sizes and defining a hierarchy of data granularities, where the small granularities are nested within larger ones. Such a hierarchy can be represented graphically as a tree.

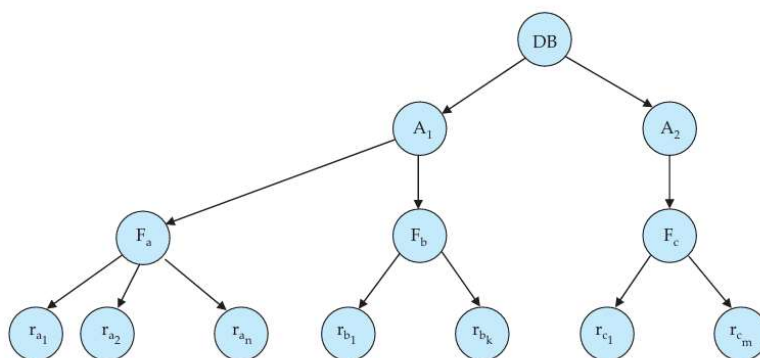


Figure 15.15 Granularity hierarchy.

Screen clipping taken: 12-06-2026 12:53

Each node in the tree can be locked individually. As we did in the two phase locking protocol, we shall use shared and exclusive lock modes. When a transaction locks a node, in either shared or exclusive mode, the transaction also has implicitly locked all the descendants of that node in the same lock mode.

Timestamp-Based Protocols

With each transaction T_i in the system, we associate a unique fixed timestamp, denoted by $TS(T_i)$. This timestamp is assigned by the database system before the transaction T_i starts execution. If a transaction T_i has been assigned timestamp $TS(T_i)$, and a new transaction T_j enters the system, then $TS(T_i) < TS(T_j)$. There are two simple methods for implementing this scheme:

1. Use the value of the **system clock** as the timestamp; that is, a transaction's timestamp is equal to the value of the clock when the transaction enters the system.
2. Use a **logical counter** that is incremented after a new timestamp has been assigned; that is, a transaction's timestamp is equal to the value of the counter when the transaction enters the system.

The timestamps of the transactions determine the serializability order

To implement this scheme, we associate with each data item Q two timestamp values:

- **W-timestamp**(Q) denotes the largest timestamp of any transaction that executed **write**(Q) successfully.
- **R-timestamp**(Q) denotes the largest timestamp of any transaction that executed **read**(Q) successfully.

These timestamps are updated whenever a new **read**(Q) or **write**(Q) instruction is executed. The timestamp-ordering protocol ensures that any conflicting read and write operations are executed in timestamp order.

1. Suppose that transaction T_i issues **read**(Q).
 - a. If $TS(T_i) < W\text{-timestamp}(Q)$, then T_i needs to read a value of Q that was already overwritten. Hence, the **read** operation is rejected, and T_i is rolled back.
 - b. If $TS(T_i) \geq W\text{-timestamp}(Q)$, then the **read** operation is executed, and $R\text{-timestamp}(Q)$ is set to the maximum of $R\text{-timestamp}(Q)$ and $TS(T_i)$.
2. Suppose that transaction T_i issues **write**(Q).
 - a. If $TS(T_i) < R\text{-timestamp}(Q)$, then the value of Q that T_i is producing was needed previously, and the system assumed that that value would never be produced. Hence, the system rejects the **write** operation and rolls T_i back.
 - b. If $TS(T_i) < W\text{-timestamp}(Q)$, then T_i is attempting to write an obsolete value of Q . Hence, the system rejects this **write** operation and rolls T_i back.
 - c. Otherwise, the system executes the **write** operation and sets $W\text{-timestamp}(Q)$ to $TS(T_i)$.

The modification to the timestamp-ordering protocol, called **Thomas' write rule**, is this: Suppose that transaction T_i issues **write**(Q).

1. If $TS(T_i) < R\text{-timestamp}(Q)$, then the value of Q that T_i is producing was previously needed, and it had been assumed that the value would never be produced. Hence, the system rejects the **write** operation and rolls T_i back.
2. If $TS(T_i) < W\text{-timestamp}(Q)$, then T_i is attempting to write an obsolete value of Q . Hence, this **write** operation can be ignored.
3. Otherwise, the system executes the **write** operation and sets $W\text{-timestamp}(Q)$ to $TS(T_i)$.

Validation-Based Protocols

Screen clipping taken: 12-06-2026 13:03

where a majority of transactions are read-only transactions, the rate of conflicts among transactions may be low. A concurrency-control scheme imposes overhead of code execution and possible delay of transactions. It may be better to use an alternative scheme that imposes less overhead.

The validation protocol requires that each transaction T_i executes in two or three different phases in its lifetime, depending on whether it is a read-only or an update transaction. The phases are, in order:

1. **Read phase.** During this phase, the system executes transaction T_i . It reads the values of the various data items and stores them in variables local to T_i . It performs all write operations on temporary local variables, without updates of the actual database.
2. **Validation phase.** The validation test (described below) is applied to transaction T_i . This determines whether T_i is allowed to proceed to the write phase without causing a violation of serializability. If a transaction fails the validation test, the system aborts the transaction.
3. **Write phase.** If the validation test succeeds for transaction T_i , the temporary local variables that hold the results of any write operations performed by T_i are copied to the database. Read-only transactions omit this phase.

Screen clipping taken: 12-06-2026 13:13

To perform the validation test, we need to know when the various phases of transactions took place. We shall, therefore, associate three different timestamps with each transaction T_i :

1. **Start(T_i)**, the time when T_i started its execution.
2. **Validation(T_i)**, the time when T_i finished its read phase and started its validation phase.
3. **Finish(T_i)**, the time when T_i finished its write phase.

Screen clipping taken: 12-06-2026 13:14

The **validation test** for transaction T_i requires that, for all transactions T_k with $TS(T_k) < TS(T_i)$, one of the following two conditions must hold:

1. $Finish(T_k) < Start(T_i)$. Since T_k completes its execution before T_i started, the serializability order is indeed maintained.
2. The set of data items written by T_k does not intersect with the set of data items read by T_i , and T_k completes its write phase before T_i starts its validation phase ($Start(T_i) < Finish(T_k) < Validation(T_i)$). This condition ensures that the writes of T_k and T_i do not overlap. Since the writes of T_k do not affect the read of T_i , and since T_i cannot affect the read of T_k , the serializability order is indeed maintained.

Screen clipping taken: 12-06-2026 13:17

Multiversion Schemes

Screen clipping taken: 12-06-2026 13:17

The concurrency-control schemes discussed thus far ensure serializability by either delaying an operation or aborting the transaction that issued the operation. A read operation may be delayed because the appropriate value has not been written yet; or it may be rejected because the value that it was supposed to read

has already been over written.

These difficulties could be avoided if old copies of each data item were kept in a system

In multiversion concurrency-control schemes, each write(Q) operation creates a new version of Q. When a transaction issues a read(Q) operation, the concurrency-control manager selects one of the versions of Q to be read.

The concurrency-control scheme must ensure that the version to be read is selected in a manner that ensures serializability

The timestamp-ordering protocol can be extended to a multiversion protocol. With each transaction T_i in the system, we associate a unique static timestamp, denoted by $TS(T_i)$.

The database system assigns this timestamp before the trans action starts execution,

With each data item Q , a sequence of versions $\langle Q_1, Q_2, \dots, Q_m \rangle$ is associated. Each version Q_k contains three data fields:

- **Content** is the value of version Q_k .
- **W-timestamp**(Q_k) is the timestamp of the transaction that created version Q_k .
- **R-timestamp**(Q_k) is the largest timestamp of any transaction that successfully read version Q_k .

Screen clipping taken: 12-06-2026 13:36

The **multiversion timestamp-ordering scheme** presented next ensures serializability. The scheme operates as follows: Suppose that transaction T_i issues a read(Q) or write(Q) operation. Let Q_k denote the version of Q whose write timestamp is the largest write timestamp less than or equal to $TS(T_i)$.

1. If transaction T_i issues a read(Q), then the value returned is the content of version Q_k .
2. If transaction T_i issues write(Q), and if $TS(T_i) < \text{R-timestamp}(Q_k)$, then the system rolls back transaction T_i . On the other hand, if $TS(T_i) = \text{W-timestamp}(Q_k)$, the system overwrites the contents of Q_k ; otherwise (if $TS(T_i) > \text{R-timestamp}(Q_k)$), it creates a new version of Q .

Screen clipping taken: 12-06-2026 13:38

15.6.2 Multiversion Two-Phase Locking

The **multiversion two-phase locking protocol** attempts to combine the advantages of multiversion concurrency control with the advantages of two-phase locking. This protocol differentiates between **read-only transactions** and **update transactions**.

Screen clipping taken: 12-06-2026 13:39

Insert Operations, Delete Operations, and Predicate Reads

Screen clipping taken: 12-06-2026 13:39

Some trans actions require not only access to existing data items, but also the ability to create new data items. Others require the ability to delete data items. To examine how such transactions affect concurrency control, we introduce these additional operations:

- delete(Q) deletes data item Q from the database.
- insert(Q) inserts a new data item Q into the database and assigns Q an initial value.

15.8.1 Deletion

To understand how the presence of **delete** instructions affects concurrency control, we must decide when a **delete** instruction conflicts with another instruction. Let I_i and I_j be instructions of T_i and T_j , respectively, that appear in schedule S in consecutive order. Let $I_i = \text{delete}(Q)$. We consider several instructions I_j .

- $I_j = \text{read}(Q)$. I_i and I_j conflict. If I_i comes before I_j , T_j will have a logical error. If I_j comes before I_i , T_j can execute the **read** operation successfully.
- $I_j = \text{write}(Q)$. I_i and I_j conflict. If I_i comes before I_j , T_j will have a logical error. If I_j comes before I_i , T_j can execute the **write** operation successfully.
- $I_j = \text{delete}(Q)$. I_i and I_j conflict. If I_i comes before I_j , T_i will have a logical error. If I_j comes before I_i , T_i will have a logical error.
- $I_j = \text{insert}(Q)$. I_i and I_j conflict. Suppose that data item Q did not exist prior to the execution of I_i and I_j . Then, if I_i comes before I_j , a logical error results for T_i . If I_j comes before I_i , then no logical error results. Likewise, if Q existed prior to the execution of I_i and I_j , then a logical error results if I_j comes before I_i , but not otherwise.

Screen clipping taken: 12-06-2026 16:36

We can conclude the following:

- Under the two-phase locking protocol, an exclusive lock is required on a data item before that item can be deleted.
- Under the timestamp-ordering protocol, a test similar to that for a **write** must be performed. Suppose that transaction T_i issues **delete**(Q).
 - If $\text{TS}(T_i) < \text{R-timestamp}(Q)$, then the value of Q that T_i was to delete has already been read by a transaction T_j with $\text{TS}(T_j) > \text{TS}(T_i)$. Hence, the **delete** operation is rejected, and T_i is rolled back.
 - If $\text{TS}(T_i) < \text{W-timestamp}(Q)$, then a transaction T_j with $\text{TS}(T_j) > \text{TS}(T_i)$ has written Q . Hence, this **delete** operation is rejected, and T_i is rolled back.
 - Otherwise, the **delete** is executed.

Screen clipping taken: 12-06-2026 16:37

15.8.2 Insertion

We have already seen that an **insert**(Q) operation conflicts with a **delete**(Q) operation. Similarly, **insert**(Q) conflicts with a **read**(Q) operation or a **write**(Q) operation; no **read** or **write** can be performed on a data item before it exists.

Since an **insert**(Q) assigns a value to data item Q , an **insert** is treated similarly to a **write** for concurrency-control purposes:

- Under the two-phase locking protocol, if T_i performs an **insert**(Q) operation, T_i is given an exclusive lock on the newly created data item Q .
- Under the timestamp-ordering protocol, if T_i performs an **insert**(Q) operation, the values **R-timestamp**(Q) and **W-timestamp**(Q) are set to $\text{TS}(T_i)$.

Screen clipping taken: 12-06-2026 16:38

=====

Recovery System

Screen clipping taken: 12-06-2026 16:51

An integral part of a database system is a recovery scheme that can restore the database to the consistent state that existed before the failure. The recovery scheme must also provide high availability;

Failure Classification:

There are various types of failure that may occur in a system,

- Transaction failure.
 - System crash
 - Disk failure
-
- **Transaction failure.** There are two types of errors that may cause a transaction to fail:
 - **Logical error.** The transaction can no longer continue with its normal execution because of some internal condition, such as bad input, data not found, overflow, or resource limit exceeded.
 - **System error.** The system has entered an undesirable state (for example, deadlock), as a result of which a transaction cannot continue with its normal execution. The transaction, however, can be reexecuted at a later time.
 - **System crash.** There is a hardware malfunction, or a bug in the database software or the operating system, that causes the loss of the content of volatile storage, and brings transaction processing to a halt. The content of nonvolatile storage remains intact, and is not corrupted.

Screen clipping taken: 12-06-2026 18:56

The assumption that hardware errors and bugs in the software bring the system to a halt, but do not corrupt the nonvolatile storage contents, is known as the **fail-stop assumption**. Well-designed systems have numerous internal checks, at the hardware and the software level, that bring the system to a halt when there is an error. Hence, the fail-stop assumption is a reasonable one.

- **Disk failure.** A disk block loses its content as a result of either a head crash or failure during a data-transfer operation. Copies of the data on other disks, or archival backups on tertiary media, such as DVD or tapes, are used to recover from the failure.

Screen clipping taken: 12-06-2026 18:56

To determine how the system should recover from failures, we need to identify the failure modes of those devices used for storing data. We can then propose algorithms to ensure database consistency and transaction atomicity despite failures. These algorithms, known as recovery algorithms, have two parts:

1. Actions taken during normal transaction processing to ensure that enough information exists to allow recovery from failures.
2. Actions taken after a failure to recover the database contents to a state that ensures database consistency, transaction atomicity, and durability.

Screen clipping taken: 12-06-2026 18:59

Storage

Screen clipping taken: 12-06-2026 19:00

we saw that storage media can be distinguished by their relative speed, capacity, and resilience to failure. We identified three categories of storage:

- **Volatile storage**
- **Nonvolatile storage**
- **Stable storage**

Screen clipping taken: 12-06-2026 19:01

In the remainder of this section, we discuss how storage media can be protected from failure during data transfer. Block transfer between memory and disk storage can result in:

- **Successful completion.** The transferred information arrived safely at its destination.
- **Partial failure.** A failure occurred in the midst of transfer, and the destination block has incorrect information.
- **Total failure.** The failure occurred sufficiently early during the transfer that the destination block remains intact.

Screen clipping taken: 12-06-2026 19:07

We require that, if a data-transfer failure occurs, the system detects it and invokes a recovery procedure to restore the block to a consistent state. To do so, the system must maintain two physical blocks for each logical database block;

1. Write the information onto the first physical block.
2. When the first write completes successfully, write the same information onto the second physical block.
3. The output is completed only after the second write completes successfully.

Screen clipping taken: 12-06-2026 19:08

storage (usually disks) and only parts of the database are in memory at any time.¹ The database is partitioned into fixed-length storage units called blocks. The blocks residing on the disk are referred to as physical blocks; the blocks residing temporarily in main memory are referred to as buffer blocks. The area of memory where blocks reside temporarily is called the disk buffer.

Block movements between disk and main memory are initiated through the following two operations:

1. $\text{input}(B)$ transfers the physical block B to main memory.
2. $\text{output}(B)$ transfers the buffer block B to the disk, and replaces the appropriate physical block there.

Screen clipping taken: 12-06-2026 19:09

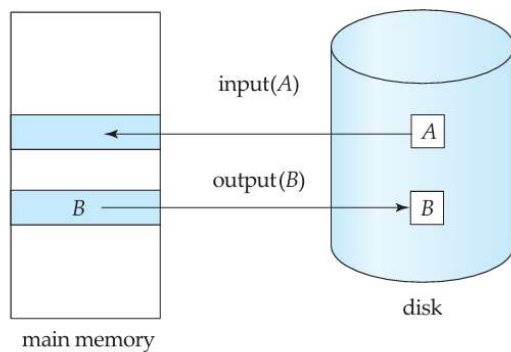


Figure 16.1 Block storage operations.

Screen clipping taken: 13-06-2026 04:21

Recovery and Atomicity

Screen clipping taken: 13-06-2026 04:24

16.3.1 Log Records

The most widely used structure for recording database modifications is the **log**. The log is a sequence of **log records**, recording all the update activities in the database.

There are several types of log records. An **update log record** describes a single database write. It has these fields:

Screen clipping taken: 13-06-2026 04:24

- **Transaction identifier**, which is the unique identifier of the transaction that performed the write operation.
- **Data-item identifier**, which is the unique identifier of the data item written. Typically, it is the location on disk of the data item, consisting of the block identifier of the block on which the data item resides, and an offset within the block.
- **Old value**, which is the value of the data item prior to the write.

Screen clipping taken: 13-06-2026 04:25

- **New value**, which is the value that the data item will have after the write.

Screen clipping taken: 13-06-2026 04:25

We represent an update log record as $\langle T_i, X_j, V_1, V_2 \rangle$, indicating that transaction T_i has performed a write on data item X_j . X_j had value V_1 before the write, and has value V_2 after the write. Other special log records exist to record significant events during transaction processing, such as the start of a transaction and the commit or abort of a transaction. Among the types of log records are:

- $\langle T_i \text{ start} \rangle$. Transaction T_i has started.
- $\langle T_i \text{ commit} \rangle$. Transaction T_i has committed.
- $\langle T_i \text{ abort} \rangle$. Transaction T_i has aborted.

Screen clipping taken: 13-06-2026 04:30

Database Modification

we need to consider the steps a transaction takes in modifying a data item:

1. The transaction performs some computations in its own private part of main memory.
2. The transaction modifies the data block in the disk buffer in main memory holding the data item.
3. The database system executes the output operation that writes the data block to disk.

Screen clipping taken: 13-06-2026 04:32

We say a transaction modifies the database if it performs an update on a disk buffer, or on the disk itself

If a transaction does not modify the database until it has committed, it is said to use the **deferred-modification** technique

If databasemodificationsoccurwhilethetransactionisstillactive,thetransaction is said to use the **immediate-modification** technique.

Because all database modifications must be preceded by the creation of a log record, the system has available both the old value prior to the modification of the data item and the new value that is to be written for the data item. This allows the system to perform *undo* and *redo* operations as appropriate.

- **Undo** using a log record sets the data item specified in the log record to the old value.
- **Redo** using a log record sets the data item specified in the log record to the new value.

Screen clipping taken: 13-06-2026 04:37

Using the Log to Redo and Undo Transactions:

We now provide an overview of how the log can be used to recover from a system crash ,and to rollback transactions during normal operation.

```
<T0 start>
<T0, A, 1000, 950>
<T0, B, 2000, 2050>
<T0 commit>
<T1 start>
<T1, C, 700, 600>
<T1 commit>
```

Figure 16.2 Portion of the system log corresponding to T_0 and T_1 .

Screen clipping taken: 13-06-2026 04:51

Figure 16.2 Portion of the system log corresponding to T_0 and T_1 .

```
T0: read(A);
    A := A - 50;
    write(A);
    read(B);
    B := B + 50;
    write(B).
```

Let T_1 be a transaction that withdraws \$100 from account C:

```
T1: read(C);
    C := C - 100;
    write(C).
```

Screen clipping taken: 13-06-2026 04:51

The recovery scheme uses two recovery procedures. Both these procedures make use of the log to find the set of data items updated by each transaction T_i , and their respective old and new values.

- **redo(T_i)** sets the value of all data items updated by transaction T_i to the new values.

more efficient since the log needs to be read only once overall, instead of once per transaction.

- **undo(T_i)** restores the value of all data items updated by transaction T_i to the old values.

- The undo operation not only restores the data items to their old value, but also writes log records to record the updates performed as part of the undo process.

These log records are special redo-only log records, since they do not need to contain the old-value of the updated data item.

- When the undo operation for transaction T_i completes, it writes a log record, indicating that the undo has completed.

Log	Database
$\langle T_0 \text{ start} \rangle$	
$\langle T_0, A, 1000, 950 \rangle$	
$\langle T_0, B, 2000, 2050 \rangle$	
	$A = 950$
	$B = 2050$
$\langle T_0 \text{ commit} \rangle$	
$\langle T_1 \text{ start} \rangle$	
$\langle T_1, C, 700, 600 \rangle$	
	$C = 600$
$\langle T_1 \text{ commit} \rangle$	

Figure 16.3 State of system log and database corresponding to T_0 and T_1 .

Screen clipping taken: 13-06-2026 04:57

After a system crash has occurred, the system consults the log to determine which transactions need to be redone, and which need to be undone so as to ensure atomicity.

- Transaction T_i needs to be undone if the log contains the record $\langle T_i \text{ start} \rangle$, but does not contain either the record $\langle T_i \text{ commit} \rangle$ or the record $\langle T_i \text{ abort} \rangle$.
- Transaction T_i needs to be redone if the log contains the record $\langle T_i \text{ start} \rangle$ and either the record $\langle T_i \text{ commit} \rangle$ or the record $\langle T_i \text{ abort} \rangle$. It may seem strange to redo T_i if the record $\langle T_i \text{ abort} \rangle$ is in the log. To see why this works, note

Screen clipping taken: 13-06-2026 04:59

Checkpoints

Screen clipping taken: 13-06-2026 05:02

When a system crash occurs, we must consult the log to determine those transactions that need to be redone and those that need to be undone. In principle, we need to search the entire log to determine this information. There are two major difficulties with this approach:

1. The search process is time-consuming.
2. Most of the transactions that, according to our algorithm, need to be redone have already written their updates into the database. Although redoing them will cause no harm, it will nevertheless cause recovery to take longer.

To reduce these types of overhead, we introduce checkpoints.

Screen clipping taken: 13-06-2026 05:02

A checkpoint is performed as follows:

1. Output onto stable storage all log records currently residing in main memory.
2. Output to the disk all modified buffer blocks.
3. Output onto stable storage a log record of the form $\langle \text{checkpoint } L \rangle$, where L is a list of transactions active at the time of the checkpoint.

Screen clipping taken: 13-06-2026 05:03

Recovery Algorithm

16.4.1 Transaction Rollback

First consider transaction rollback during normal operation (that is, not during recovery from a system crash). Rollback of a transaction T_i is performed as follows:

1. The log is scanned backward, and for each log record of T_i of the form $\langle T_i, X_j, V_1, V_2 \rangle$ that is found:
 - a. The value V_1 is written to data item X_j , and
 - b. A special redo-only log record $\langle T_i, X_j, V_1 \rangle$ is written to the log, where V_1 is the value being restored to data item X_j during the rollback. These log records are sometimes called **compensation log records**. Such records do not need undo information, since we never need to undo such an undo operation. We shall explain later how they are used.
2. Once the log record $\langle T_i \text{ start} \rangle$ is found the backward scan is stopped, and a log record $\langle T_i \text{ abort} \rangle$ is written to the log.

Screen clipping taken: 13-06-2026 05:08

16.4.2 Recovery After a System Crash

Recovery actions, when the database system is restarted after a crash, take place in two phases:

Screen clipping taken: 13-06-2026 05:08

1. In the **redo phase**,

Screen clipping taken: 13-06-2026 05:10

2. In the **undo phase**,

Screen clipping taken: 13-06-2026 05:11

1. In the **redo phase**, the system replays updates of *all* transactions by scanning the log forward from the last checkpoint. The log records that are replayed include log records for transactions that were rolled back before system crash, and those that had not committed when the system crash occurred.

Screen clipping taken: 13-06-2026 05:11

- a. The list of transactions to be rolled back, undo-list, is initially set to the list L in the $\langle \text{checkpoint } L \rangle$ log record.

16.4 Recovery Algorithm 737

- b. Whenever a normal log record of the form $\langle T_i, X_j, V_1, V_2 \rangle$, or a redo-only log record of the form $\langle T_i, X_j, V_2 \rangle$ is encountered, the operation is redone; that is, the value V_2 is written to data item X_j .
- c. Whenever a log record of the form $\langle T_i \text{ start} \rangle$ is found, T_i is added to undo-list.
- d. Whenever a log record of the form $\langle T_i \text{ abort} \rangle$ or $\langle T_i \text{ commit} \rangle$ is found, T_i is removed from undo-list.

Screen clipping taken: 13-06-2026 05:12

- 2. In the **undo phase**, the system rolls back all transactions in the undo-list. It performs rollback by scanning the log backward from the end.
 - a. Whenever it finds a log record belonging to a transaction in the undo-list, it performs undo actions just as if the log record had been found during the rollback of a failed transaction.
 - b. When the system finds a $\langle T_i \text{ start} \rangle$ log record for a transaction T_i in undo-list, it writes a $\langle T_i \text{ abort} \rangle$ log record to the log, and removes T_i from undo-list.
 - c. The undo phase terminates once undo-list becomes empty, that is, the system has found $\langle T_i \text{ start} \rangle$ log records for all transactions that were initially in undo-list.

Screen clipping taken: 13-06-2026 05:13

Failure with Loss of Nonvolatile Storage

Screen clipping taken: 13-06-2026 08:06

The basic scheme is to dump the entire contents of the database to stable storage periodically—say, once per day

If a failure occurs that results in the loss of physical database blocks, the system uses the most recent dump in restoring the database to a previous consistent state.

Once this restoration has been accomplished, the system uses the log to bring the database system to the most recent consistent state.

One approach to database dumping requires that no transaction may be active during the dump procedure, and uses a procedure similar to checkpointing:

1. Output all log records currently residing in main memory onto stable storage.
2. Output all buffer blocks onto the disk.
3. Copy the contents of the database to stable storage.
4. Output a log record $\langle \text{dump} \rangle$ onto the stable storage.

Screen clipping taken: 13-06-2026 08:10

To recover from the loss of nonvolatile storage, the system restores the database to disk by using the most recent dump. Then, it consults the log and redoes all the actions since the most recent dump occurred. Notice that no undo operations need to be executed.

A dump of the database contents is also referred to as an archival dump,

Remote Backup Systems

Screen clipping taken: 13-06-2026 08:12

We can achieve high availability by performing transaction processing at one site, called the **primary site**, and having a **remote backup site** where all the data from the primary site are replicated.

The remote backup site is sometimes also called the **secondary site**.

The remote site must be kept synchronized with the primary site, as updates are performed at the primary. We achieve synchronization by sending all log records from primary site to the remote backup site

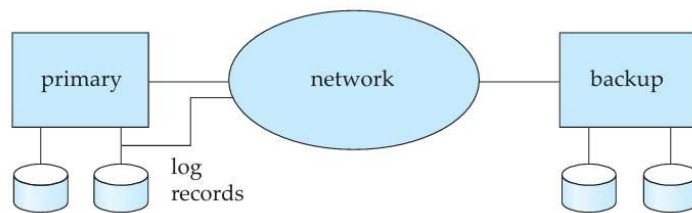


Figure 16.10 Architecture of remote backup system.

Screen clipping taken: 13-06-2026 08:23

When the primary site fails, the remote backup site takes over processing. First, however, it performs recovery, using its (perhaps outdated) copy of the data from the primary, and the log records received from the primary.

the remote backup site is performing recovery actions that would have been performed at the primary site when the latter recovered. Standard recovery algorithms, with minor modifications, can be used for recovery at the remote backup site. Once recovery has been performed, the remote backup site starts processing transactions.

Several issues must be addressed in designing a remote backup system:

- Detection of failure.
- Transfer of control.
- Time to recover.
- Time to commit.
 - One-safe.
 - Two-very-safe.
 - Two-safe.

- Detection of failure.

It is important for the remote backup system to detect when the primary has failed. Failure of communication lines can fool the remote backup into believing that the primary has failed.

- Transfer of control.

When the primary fails, the backup site takes over processing and becomes the new primary. When the original primary site recovers, it can either play the role of remote backup, or take over the role of primary site again. In either case, the old primary must receive a log of updates carried out by the backup site while the old primary was down.

- Time to recover.

If the log at the remote backup grows large, recovery will take a long time. The remote backup site can periodically process the redo log records that it has received and can perform a checkpoint, so that earlier parts of the log can be deleted

